

Learning System 6: Support Vector Machine(SVM)

Classification

DT8008 VT26, Halmstad University

Binay Kumar Sah, binsah25@student.hh.se, Master's in Embedded and Intelligence System
Anton Bauer, antibau25@student.hh.se, TAEIS 25

March 2026

Introduction

In this lab we explore binary classification using Support Vector Machines (SVM) to understand both linear and non-linear decision boundaries. In Part A, we implemented a Gaussian kernel and performed a grid search to optimize the hyperparameters C and σ , and observed their impacts. In part B we applied these concepts to a real-world spam classifier, involving email pre-processing, feature extraction via a vocabulary list, and high-dimensional data analysis.

Part A: Support Vector Machine

Objective:

Part A objective was to develop a model using Support Vector Machine(SVM) and explore how they handle both linear and non-linear data. Later we also analyzed how the different value of C and σ control the penalty for misclassifications and classification curve for non linear data respectively.

Key Concept:

1. **Support Vector Machine:** SVM is a supervised machine learning algorithm used primarily for binary classification by finding the maximum possible margin. The points that lie closest to this boundary and define its position are called "support vectors".

SVM address the "Outlier" problems by introducing the margin to ignore noises.

Additionally, using a kernel trick SVM projects data into a higher-dimensional space where a complex, curvy boundary can be drawn to separate intertwined classes that a straight line could never handle.

2. **The C parameter:** This is a regularization value that controls the trade-off between maximizing the margin and minimizing classification errors.
 - **High C :** Penalizes misclassifications heavily, often leading to a complex, overfitted boundary.
 - **Low C :** Allows for a simpler boundary that may ignore outliers.
3. **Gaussian Kernel (RBF):** A similarity function that was used to handle non-linearly separable data by measuring the "distance" between pairs of examples. It effectively allowed the SVM to learn complex, curvy decision boundaries by mapping data into a higher-dimensional space.

4. **Sigma (σ) Hyperparameter:** This bandwidth parameter determines how quickly the influence of a single training example falls off.

- **Small σ :** Lead to high sensitivity and wiggly boundaries (overfitting)
- **large σ :** Create smoother, simpler boundaries (underfitting)

Methods/Algorithm:

Algorithm 1 Non-linear SVM with Gaussian Kernel and Hyperparameter Tuning

- 1: **Step 1: Data Loading and Linear Exploration**
 - 2: Load the 2D dataset and visualize it using a scatter plot to observe class distribution. Test a linear SVM and observe how the C parameter affects the boundary in the presence of outliers.
 - 3: **Step 2: Gaussian Kernel Implementation**
 - 4: Implement the similarity function (Gaussian Kernel) to handle non-linearly separable data.
 - 5: $K_{\text{gaussian}}(u, v) = \exp\left(-\frac{\|u-v\|^2}{2\sigma^2}\right)$
 - 6: **Step 3: Define Training Function**
 - 7: Create a helper function that converts the bandwidth σ to the internal parameter $\gamma = \frac{1}{2\sigma^2}$ and trains the model using the Radial Basis Function (RBF) kernel.
 - 8: **Step 4: Hyperparameter Grid Search**
 - 9: Iterate through a multiplicative range of values for both C and σ (e.g., $[0.01, 0.03, 0.1, 0.3, 1, 3, 10, 30]$) to find the optimal combination.
 - 10: **Step 5: Model Evaluation on Validation Set**
 - 11: For each (C, σ) pair, train on the training set (X, y) and generate predictions on the validation set (X_{val}) .
 - 12: Calculate validation accuracy: $\text{Accuracy} = \text{np.mean}(\text{predictions} == y_{val}) \times 100$.
 - 13: **Step 6: Optimal Parameter Selection**
 - 14: Identify the C_{best} and σ_{best} that result in the highest accuracy on the validation set to ensure the model generalizes well to unseen data.
 - 15: **Step 7: Final Visualization**
 - 16: Re-train the SVM using C_{best} and σ_{best} and plot the non-linear decision boundary to verify it follows the contours of the data without overfitting.
-

Observation and Results:

SVM - Linear Classification:

We use the SVM to classify the *lab5data1.mat* which can be classified by the linear line but it has one outlier data on it which impacted the linear classification. We performed the SVM at different values of C as shown in the figure 1.

Here, we can see with cost for misclassification ($C = 1$) fig 1a, the SVM prioritizes a wide and simple margin. Basically, it classifies the gap well but accepts the outlier as an error and since it has very less margin, there is high chance that it may perform poor with outlier data in unseen data. On the other hand as C increases to 10 the decision boundary shifts towards the outlier to narrow its error, resulting in a tighter margin while at high value $C = 100$ the model perfectly classified the data but it overfit to the outlier and thus it will likely perform poor in unseen data.

SVM with Gaussian Kernels

In this section we used Gaussian Kernels (RBF) to classify a dataset 2, that was not linearly separable. We observed the classification at different parameter values σ as shown in the figure 2.

Here, we found that with a very small bandwidth ($\sigma = 0.01$) the boundary was highly influenced by each local point. This creates a highly complex, "wiggly" boundary that captures every individual

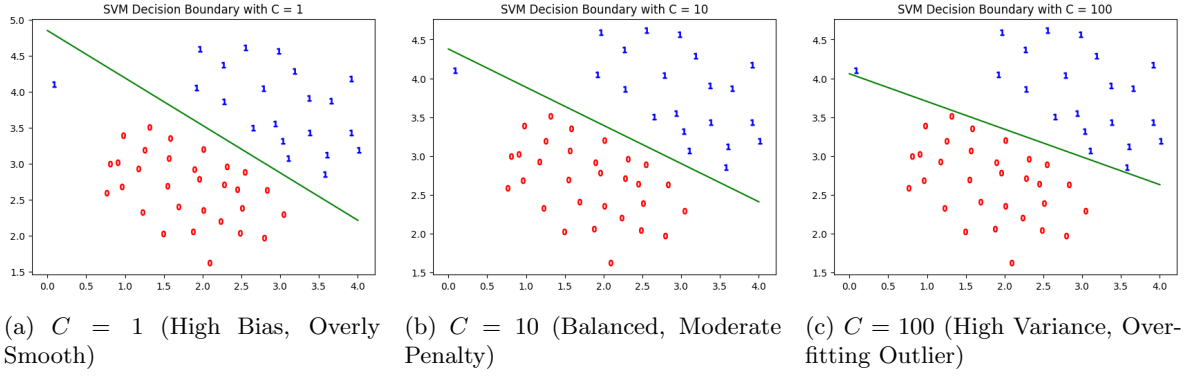


Figure 1: Visualizing the impact of the regularization parameter C on the Linear SVM decision boundary for Dataset 1. Observe how increasing C forces the green decision boundary to adjust dramatically to eliminate the single outlier (at top left) at the cost of margin width and boundary complexity.

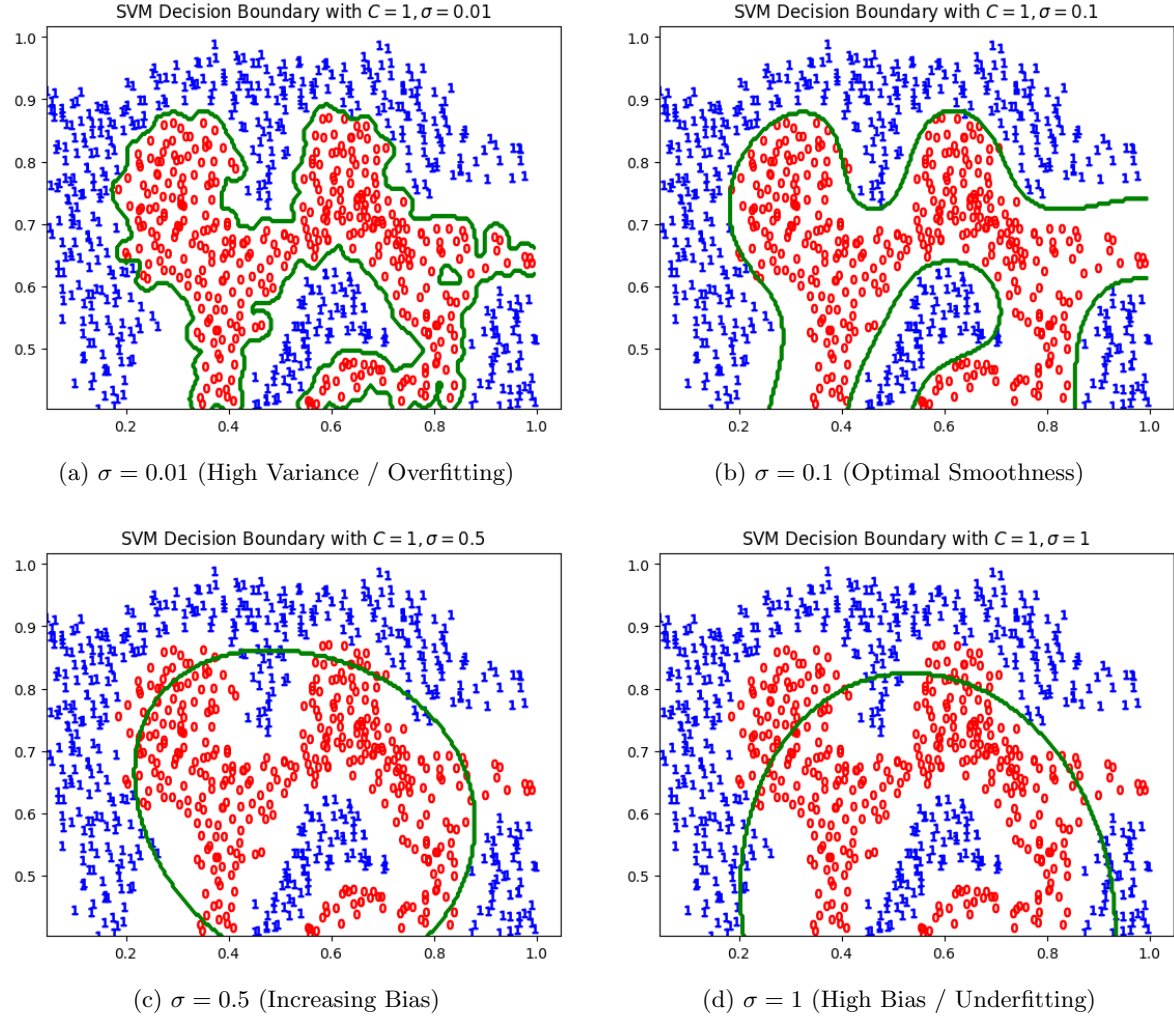


Figure 2: Evolution of the SVM decision boundary on Dataset 2 as the Gaussian kernel bandwidth (σ) increases. Note how the boundary transitions from a highly complex shape that overfits the training data to a simplified, smooth curve that eventually fails to capture the non-linear pattern.

point perfectly including the vacant region in the zone of negative red data as can be seen in fig 2a, this shows this model has less generalization.

On the other hand, at $\sigma = 0.1$, as shown in figure 2b provides the best balance for the example data. The decision boundary is smooth and separates the two classes effectively while maintaining a reasonable safety margin.

At last we also observe the impact on increasing σ for 0.5 or larger. As the bandwidth increases the influence of each point spreads out too far. This results in a boundary that is "too smooth" and fails to capture the underlying non-linear pattern of the data, leading to underfitting where many points are misclassified. Moreover, for increasing the value to 1 begins overshooting the decision boundary as can be seen in figure 2d.

Part B: SPAM Classification using SVM

Objective:

The objective of Part B was to build an email spam filter using Support Vector Machine. This involved preprocessing raw email text to a more usable format, such as URLs/numbers/dollar-signs being replaced by generic english representation. Then using that to create high-dimensional feature vector/vectors based on a frequent email words vocabulary, followed by training a linear SVM on said extracted email word data. The SVM was then used to predict spam, and to analyze its model weights to get insight into what the most predictive spam words were.

Key Concepts:

1. **Data Preprocessing and Normalization:** Transforming raw emails by stripping HTML, lower-casing, stemming, and replacing specific parts (URLs, numbers, email addresses, dollar signs) with generic tags (like "httpaddr", "dollar numb"). This prevents the model from overfitting to specific URLs or specific dollar amounts, forcing it to look at the general structure of the spam instead.
2. **Feature Extraction:** Mapping the cleaned email against an already existing vocabulary list of 1899 high-frequency spam words. Each email is represented as a vector $x \in \mathbb{R}^{1899}$ where the j^{th} element is 1 if the word is present and 0 otherwise.
3. **Principal Component Analysis (PCA):** A dimensionality reduction technique briefly used in lab to visualize the complex 1899-dimensional dataset in a 2D scatter plot. Further knowledge of inner-workings was stated not necessary for lab.

Methods/Algorithm:

Algorithm 2 Spam Classification using a Linear SVM

- 1: **Step 1: Preprocess Email Text**
 - 2: Normalize/Preprocess the raw text by converting to lowercase, removing HTML, stemming words, and replacing numbers/URLs/money with standard plain english strings.
 - 3: **Step 2: Feature Vector Extraction**
 - 4: Compare the processed email words against the pre-existing vocabulary of 1899 frequent spam words (using a fast hash-set lookup) to generate a binary vector $x \in \mathbb{R}^{1899}$.
 - 5: **Step 3: Model Training**
 - 6: Load the given pre-extracted training dataset (X, y) consisting of 4000 examples. Train a Linear SVM classifier using a regularization parameter of $C = 0.03$.
 - 7: **Step 4: Model Evaluation**
 - 8: Use the trained model to predict classifications on the also given 1000-example test dataset (X_{test}, y_{test}) . Calculate accuracies by computing the percentage of correct predictions.
 - 9: **Step 5: Identify Top Predictors**
 - 10: Extract the model parameters θ and sort the parameters to find the indices with the largest positive values, revealing the words the classifier found most indicative of spam.
-

Observation and Results:

Model Accuracy and High-Dimensionality

Because the data had 1899 dimensions, a direct visualization of the decision boundary through plotting was impossible. The lab instead showcased utilizing PCA to compress the data into 2 components, allowing for a rough visual representation of the class separation as compression loses some information. Showing that, in Fig 3, an underlying boundary between spam and normal email seems to exist.

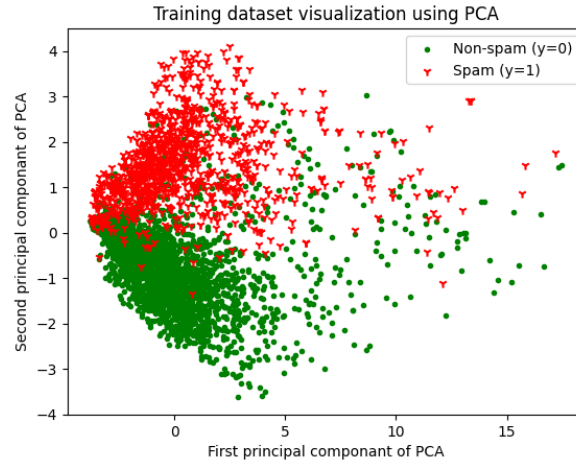


Figure 3: PCA visualization from compressing the 1899-dimensional email dataset into 2 principal components to show class separation (spam and not spam).

Training the Linear SVM with the lab stated $C = 0.03$ resulted in a good model, getting a training accuracy of 99.425% and a test accuracy of 99.0%. Although we did not find this C ourselves, we should still highlight that the high test accuracy proves that the SVM generalized well to unseen emails (not overfitting the training data).

Top Spam Predictors

Inspecting the θ_j parameter values learned by the linear model provided the features that most heavily weighted an email toward the $y = 1$ spam classification. The top 10 spam predictors (matching lab description's expectations) were : "click", "remov", "basenumb", "our", "guarante", "visit", "pleas", "you", "dollar", and "here". These words make sense logically as common spam strategies would probably urge the user to "click here", "visit", or mentioning "dollars" and "guarantees".

Custom Email Testing

To validate the model (and for fun) in a real-world scenario, we tested a custom promotional email from Ubisoft. The classifier successfully predicted the output as NON-SPAM, proving its robustness against standard marketing materials, even if they contain attention grabbing words like: "discount", "premium", "limited" and "Join now".

Conclusion

This lab helped us see the usability of Support Vector Machines. In Part A, we saw how the C parameter decides the bias-variance tradeoff (margin size vs outlier penalization) and how the Gaussian (RBF) kernel with a tuned σ bandwidth can be used to map and separate non-linear data patterns. In Part B, we saw that even a simple Linear SVM is very usable and computationally efficient for text classification when dealing with high-dimensional feature spaces, like our 1899 spam word vocabulary model.